

TALLER DE PROGRAMACION



Cómo vamos a Trabajar

Planteo de un tema

→ Conceptos teóricos

Ejercicios prácticos

Conclusiones

Formulario de preguntas

Conceptos de la clase de hoy

1

Arbol

2

ABB

características

3

ABB

operaciones

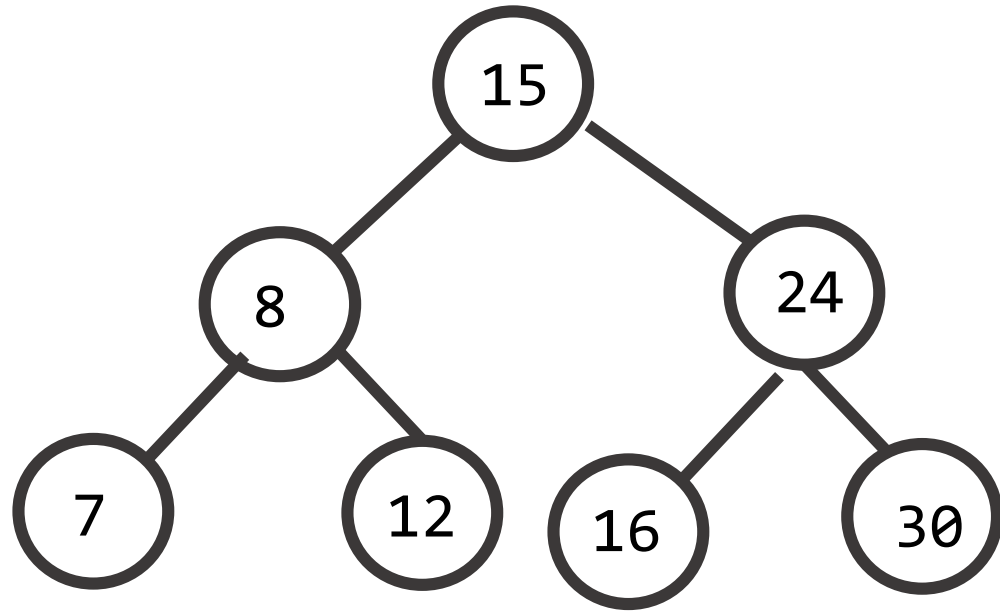
Carga

Recorridos

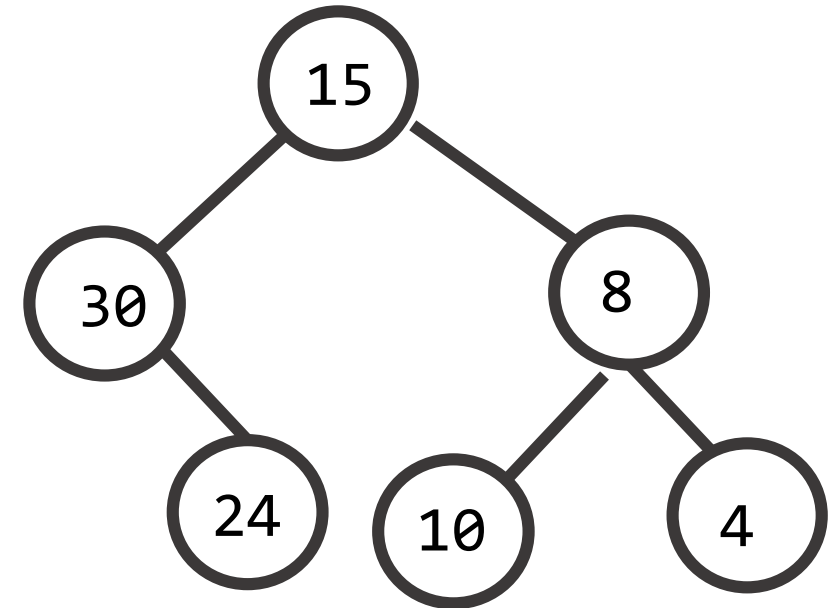
4

Conclusiones

Conceptos iniciales- Arbol Binario de Búsqueda



Todos los nodos ubicados a la izquierda de su padre son menores y todos los de la derecha son mayores.



Todos los nodos ubicados a la derecha de su padre son menores y todos los de la izquierda son mayores.

Conceptos iniciales- Arbol Binario de Búsqueda

Programa arboles;

Type

```
arbol = ^nodo;  
tipo = ...;  
nodo = record  
    dato: tipo;  
    HI: arbol;  
    HD: arbol;  
end;
```

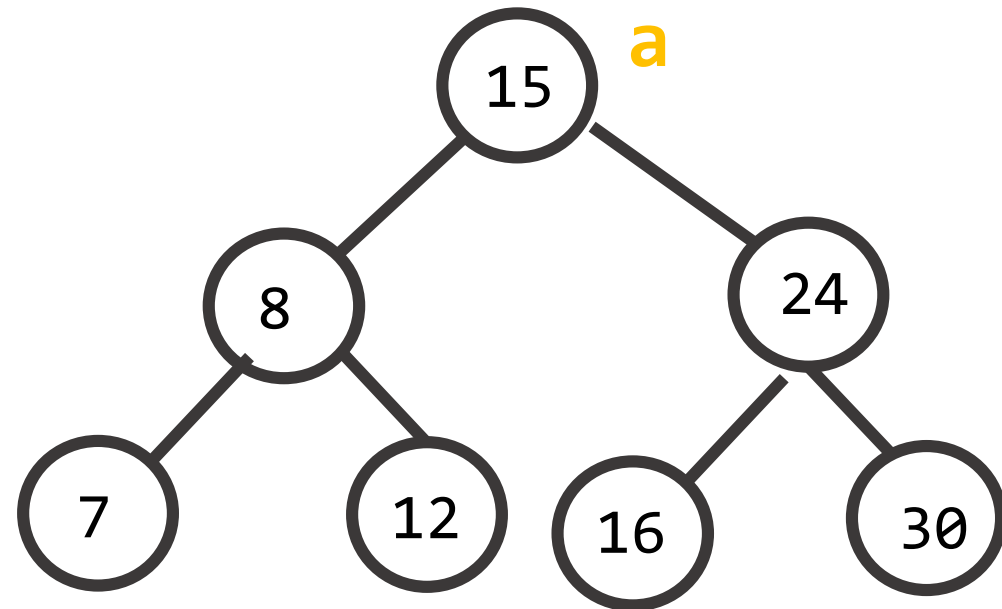
Var

a:arbol;

Begin

...

End.



Conceptos iniciales- Arbol Binario de Búsqueda

Programa arboles;

Type

arbol = ^nodo;

tipo = ...;

nodo = record

dato: tipo;

HI: arbol;

HD: arbol;

end;

Var

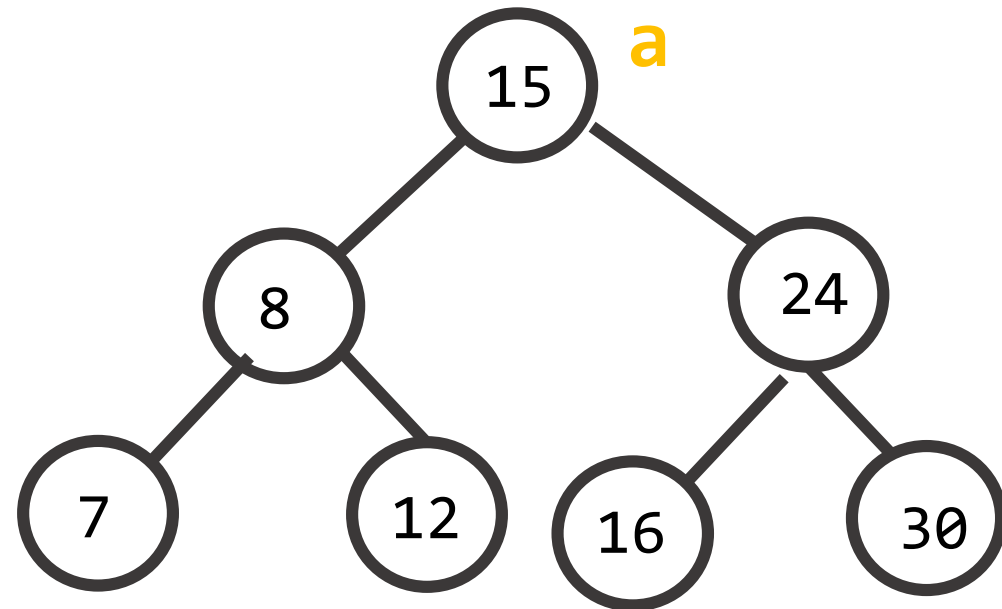
a:arbol;

Begin

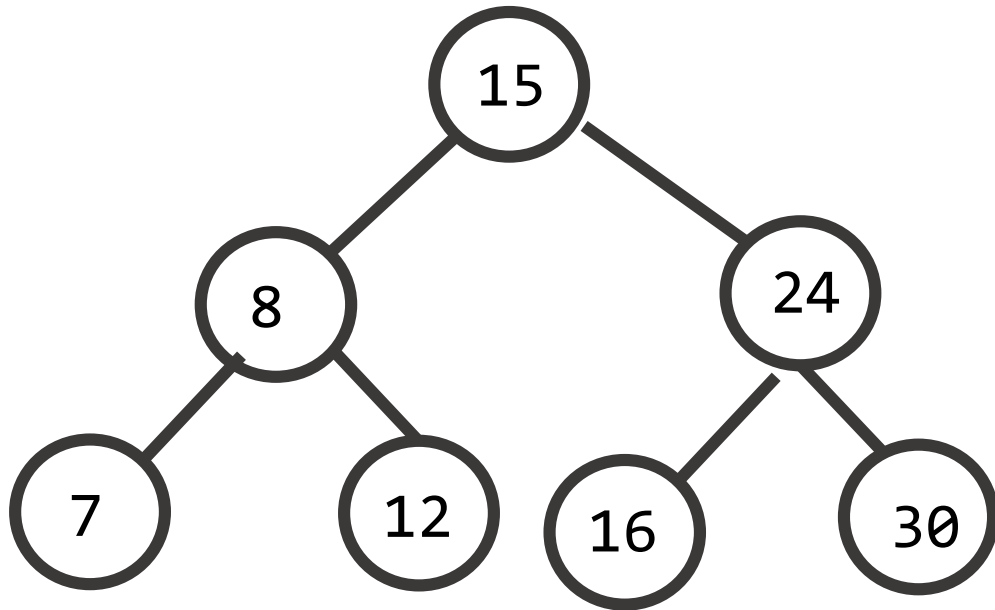
a:= nil;

crear (a);

End.



Arbol Binario de Búsqueda - IMPRESION



Cómo lo recorro?

Programa arboles;

Type

arbol = ^nodo;

nodo = record

dato: integer;

HI: arbol; HD: arbol;

end;

Var

a:arbol;

Begin

a:= nil;

read (num);

while (num <> 0) do

agregar (a,num);

read(num);

end;

enOrden(a);

End.

Arbol Binario de Búsqueda - IMPRESION

Procedure **enOrden** (a:arbol);

begin

if (a<> nil) then begin

1 **enOrden** (a^.HI);

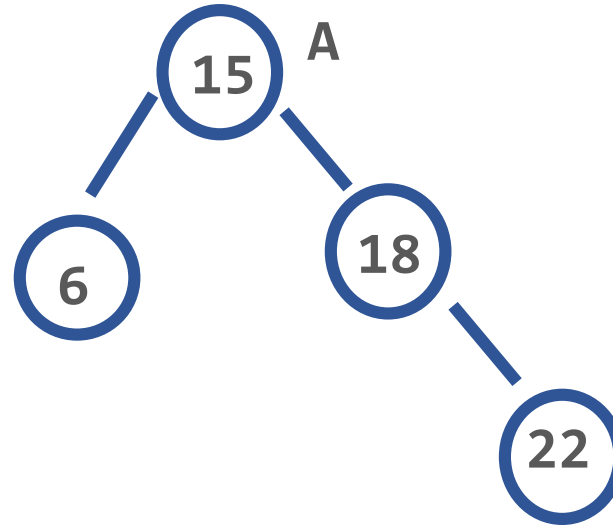
2 write (a^.dato);

3 **enOrden** (a^.HD);

end;

end;

Observar la cantidad de
llamados recursivos?



a=15
HI=6
HD=18

1 2 3

Imprime 15

a=6
HI=nil
HD=nil

1 2 3

Imprime 6

a=18
HI=nil
HD=nil

1 2 3

Imprime 18

a=nil **No**
HI=nil **hace**
HD=nil **nada**

a=nil **No**
HI=nil **hace**
HD=nil **nada**

a=nil **No**
HI=nil **hace**
HD=nil **nada**

a=22
HI=nil **1 2 3**
HD=nil

Imprime 22

a=nil **No**
HI=nil **hace**
HD=nil **nada**

a=nil **No**
HI=nil **hace**
HD=nil **nada**

Ordenación- CONCLUSIONES



Los arboles binarios de búsqueda ABB deben ser creados respetando un orden.

Los recorridos pueden hacerse de menor a mayor, de mayor a menor o para cada nodo sus hijos. (**enOrden**, **postOrden** e **preOrden**), sólo deben acomodarse los llamados para implementarlos.

Los recorridos también pueden adaptarse para otras acciones (no sólo imprimir los nodos).

Formulario